MIDTERM EXAM
DATABASE PROJECT

OIM6301 – PROGRAMMING FOR BUSINESS ANALYTICS

Mamadou NDIAYE

Table des matières

Mid Term Exam – Data Base Project (Scenario B) 2

A. **Phase 1 - Design** an ERD and draft your schema 2

1. Database design (entities, keys, relationships) 2
2. ERD schema 3
3. README – **Scenario B (Babson Investment Club)** 3

B. **Phase 2 – Data generation: Generate** CSV files with Mockaroo 3

1. Volumes & import order 3
2. Schemas 4
3. Constraints & design choices 6
4. Quick quality checks (Phase 3 preview) 6
5. README (Phase 2) 6

C. **Phase 3 – Build** a SQLite database in DBeaver and import CSV files 7

1. Create the SQLite DB and import CSV 7
2. schema.sql – Table creation 8
3. Quick data quality checks 9

D. **Phase 4 –** Business analysis (queries) 10

E. Reflection 10

Mid Term Exam – Data Base Project (Scenario B)

I chose Scenario B – Babson Investment Club (BIC) because it offers a realistic and pedagogically rich framework for applying relational database design to a dynamic student-led environment. The structure is compact yet meaningful, with four core entities—members, portfolios, assets, and trades—that reflect real-world investment workflows and support insightful analytics such as trading behavior, portfolio composition, and performance estimation. Phase 1 - Design an ERD and draft your schema

1. Database design (entities, keys, relationships)

Entities

- **Members** — member_id (PK), first_name, last_name, major, join_date
- **Portfolios** — portfolio_id (PK), member_id (FK→Members), portfolio_name, created_date, notes
- **Assets** — asset_id (PK), symbol (**UNIQUE**), asset_type (STOCK/CRYPTO/FUND), market, current_price
- **Trades** — trade_id (PK), portfolio_id (FK→Portfolios), asset_id (FK→Assets), trade_date, trade_type (BUY/SELL), quantity (>0), trade_price (>=0), fee (>=0), notes

Relationships & cardinalities

- **Members 0..M ⟶ 1 Portfolios** (a member may have zero or many portfolios; each portfolio has exactly one owner)
- **Portfolios 0..M ⟶ 1 Trades** (a portfolio may have zero or many trades; each trade belongs to one portfolio)
- **Assets 0..M ⟶ 1 Trades** (an asset may appear in zero or many trades; each trade references one asset)

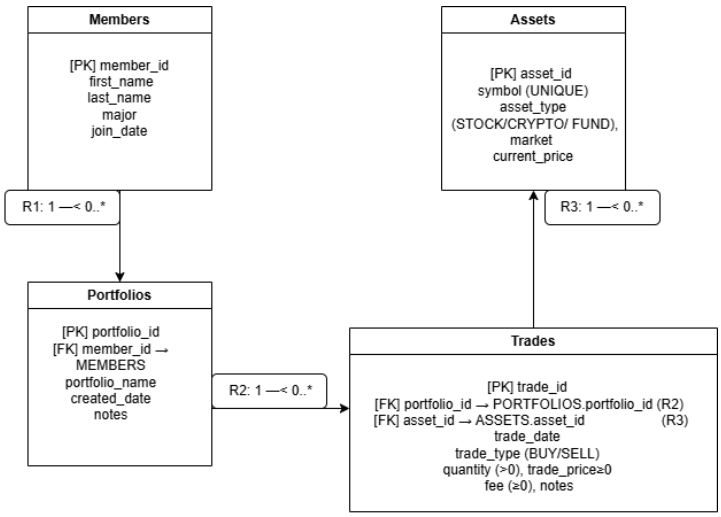
Referential integrity choices (RI)

- **Members → Portfolios: Enforce, NO CASCADE** (preserve ownership history)
- **Portfolios → Trades: Enforce, NO CASCADE** (preserve audit trail)
- **Assets → Trades: Enforce, NO CASCADE** (preserve audit trail)

Key assumptions

- A member may have zero portfolios (new or inactive members).
- An asset may exist without trades (tracked universe).
- Dates are stored as **TEXT** (YYYY-MM-DD) in SQLite.
- P&L queries are **approximate** (average cost); **no** FIFO/LIFO and **no** corporate actions.

2. ERD schema




```
Legend:
[PK] = primary key   ·   [FK] = foreign key
Cardinality: 1 —< 0..* = one-to-many   ·   0..1 —< 0..* = optional-to-many
RI policy: R1-R3 enforced with NO CASCADE (history preserved)

R1: Members 1 —< Portfolios   (one member, many portfolios)
R2: Portfolios 1 —< Trades    (one portfolio, many trades)
R3: Assets 1 —< Trades        (one asset, many trades)
```

3. README – **Scenario B (Babson Investment Club)**

The database models members, the portfolios they own, tradable assets (stocks/crypto/funds), and trades.
Relationships: Member→Portfolio (0..M:1), Portfolio→Trade (0..M:1), Asset→Trade (0..M:1).

Keys: member_id, portfolio_id, asset_id, trade_id.

Constraints: Assets.symbol is UNIQUE; Trades.trade_type ∈ {BUY, SELL}; quantity > 0, trade_price ≥ 0, fee ≥ 0.

Referential integrity: foreign keys enforced with NO CASCADE to preserve the audit trail.

What this enables: monthly trade summaries, portfolio holdings (buys − sells), top assets by activity/value, and simple P&L (approximate via average cost vs. current price; no FIFO/LIFO, no shorts).

A. Phase 2 – Data generation: Generate CSV files with Mockaroo

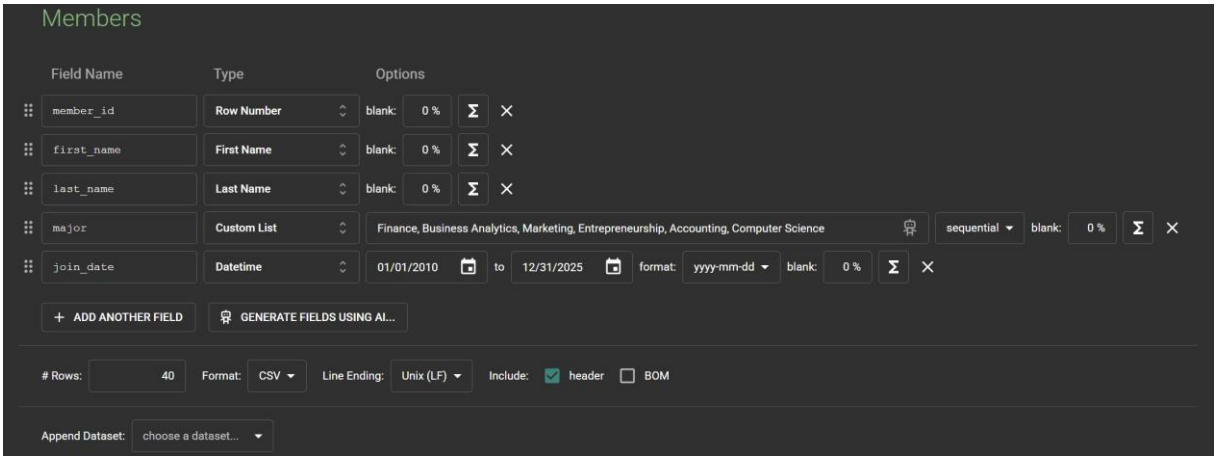
1. Volumes & import order

- **Members:** 40 rows
- **Portfolios:** 60 rows (~1–3 per member)
- **Assets:** 120 rows
- **Trades:** 800 rows
- **Import order:** Members → Portfolios → Assets → Trades (parents before children)

2. Schemas

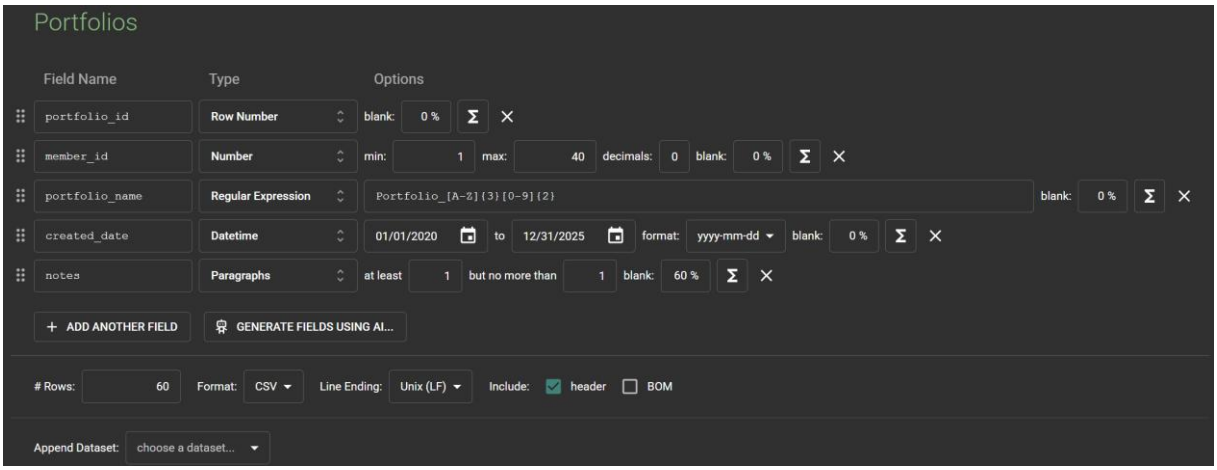
Members.csv

- member_id Row Number (1..40)
- first_name First Name · last_name Last Name · major Custom List (Finance, Business Analytics, Marketing, Entrepreneurship, Accounting, Computer Science)
- join_date Date 2010-01-01 → 2025-12-31



Portfolios.csv

- portfolio_id Row Number · member_id Number (1..40) **FK**
- portfolio_name RegEx Portfolio_[A-Z]{3}[0-9]{2}
- created_date Date 2010-01-01 → 2025-12-31
- notes Paragraphs (1 sentence) **Blank ≈ 60%**



Assets.csv

- asset_id Row Number · symbol Custom List (unique)
- asset_type Custom List (weighted): STOCK (70), CRYPTO (20), FUND (10)
- market Formula (Ruby):

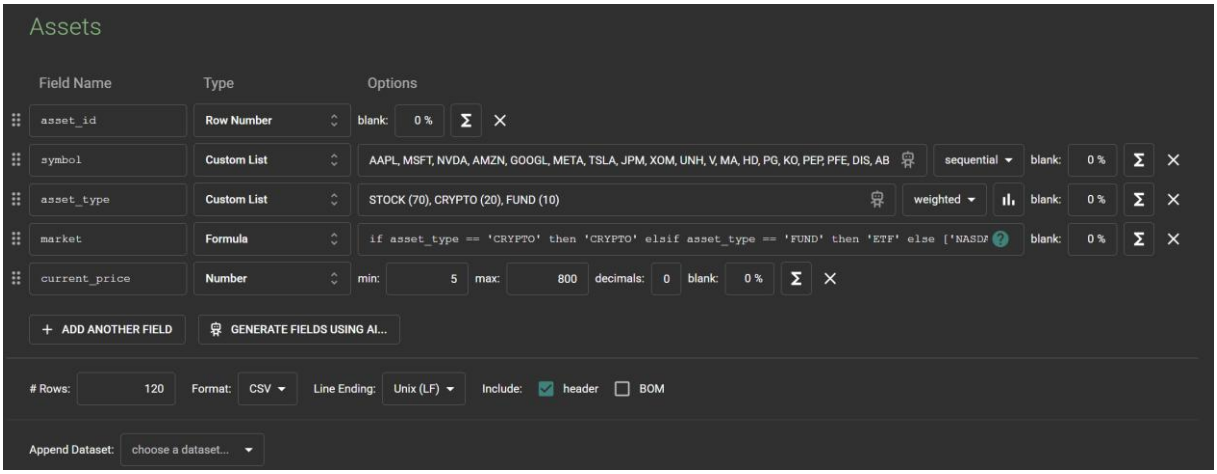
if asset_type == 'CRYPTO' then 'CRYPTO'

elsif asset_type == 'FUND' then 'ETF'

else ['NASDAQ','NYSE'].sample

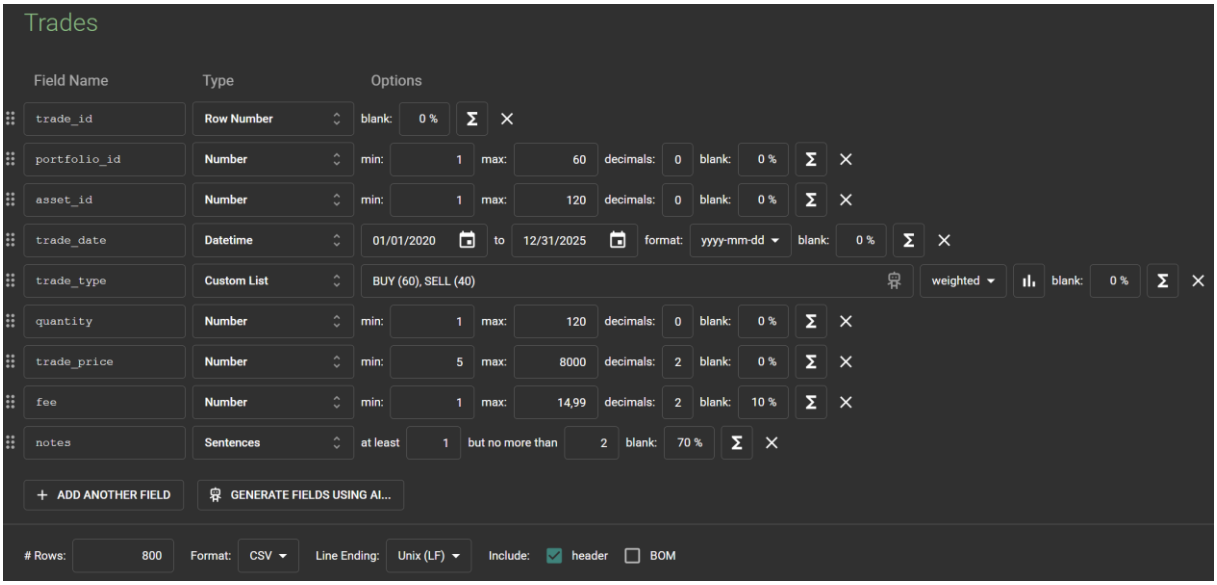
end

current_price Number (min 5, max 800, 2 decimals)



Trades.csv

- trade_id Row Number · portfolio_id Number (1..60) **FK** · asset_id Number (1..120) **FK**
- trade_date Date 2012-01-01 → 2025-12-31
- trade_type Custom List (weighted): **BUY (60)**, **SELL (40)**
- quantity Number (1..200, 0 decimals) · trade_price Number (5..8000, 2 decimals)
- fee Number (0..14.99, 2 decimals, **Blank 10%**) · notes Sentences (1–2, **Blank 70%**)



3. Constraints & design choices

- **Stable FKs:** parent tables use **Row Number** IDs (1..N); child tables use matching numeric ranges → avoids orphan rows on import.
- **Business uniqueness:** Assets.symbol unique in generation (and can be enforced UNIQUE in SQL).
- **Enums:** asset_type ∈ {STOCK, CRYPTO, FUND}; trade_type ∈ {BUY, SELL}.
- **Simplicity first:** no cross-dataset formulas; realism can be added later if desired.

4. Quick quality checks (Phase 3 preview)

```
-- Row counts
SELECT COUNT(*) FROM Members;
SELECT COUNT(*) FROM Portfolios;
SELECT COUNT(*) FROM Assets;
SELECT COUNT(*) FROM Trades;
-- Orphans (should be zero rows)
SELECT p.portfolio_id
FROM Portfolios p LEFT JOIN Members m ON m.member_id = p.member_id
WHERE m.member_id IS NULL;
-------------/
SELECT t.trade_id
FROM Trades t LEFT JOIN Portfolios p ON p.portfolio_id = t.portfolio_id
WHERE p.portfolio_id IS NULL;
SELECT t.trade_id
FROM Trades t LEFT JOIN Assets a ON a.asset_id = t.asset_id
WHERE a.asset_id IS NULL;
-- (Optional) Portfolios created before member joined
SELECT p.portfolio_id, p.member_id, p.created_date, m.join_date
FROM Portfolios p
JOIN Members m ON m.member_id = p.member_id
WHERE date(p.created_date) < date(m.join_date);
```

5. README (Phase 2)

We used Mockaroo to generate **40 Members**, **60 Portfolios**, **120 Assets**, and **800 Trades**.

Parent entities use Row Number IDs (1..N) and child CSVs use matching numeric ranges to avoid orphan foreign keys.

Members' join_date spans **2010-01-01 to 2025-12-31**.

Assets include a unique symbol, a weighted asset_type (stock/crypto/fund), and a derived market via a simple formula.

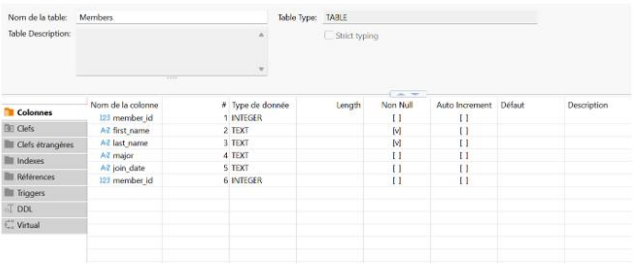
Trades use a 60/40 BUY/SELL split, integer quantities, 2-decimal prices, small fees (10% nulls), and sparse notes. We verified integrity with orphan checks after import.

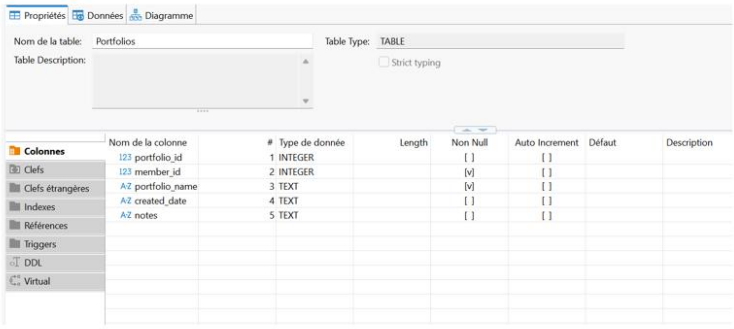
B. Phase 3 – Build a SQLite database in DBeaver and import CSV files

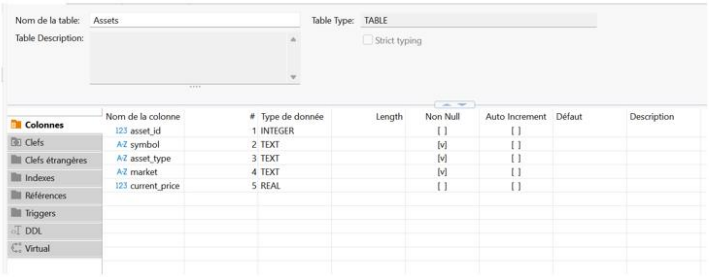
1. Create the SQLite DB and import CSV

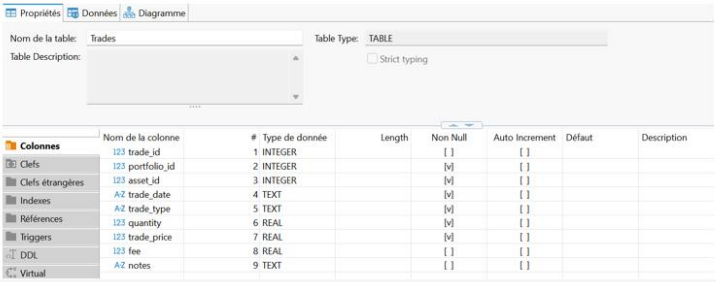
After creating the SQLite database named **MND_MidTerm_Project** in DBeaver (Database → New Database Connection → SQLite → Create → Test Connection → Finish), I proceeded to import the CSV tables generated via Mockaroo. The files were loaded in the following order: **Members.csv**, **Portfolios.csv**, **Assets.csv**, and **Trades.csv**, each mapped to its corresponding table.

For each import (import order: Members → Portfolios → Assets → Trades (parents before children), I used the *Import Data* function by right-clicking on the target table, selecting the appropriate CSV file, configuring the encoding (UTF-8) and separator (,), and verifying that each column was correctly mapped. Once the data was loaded, the tables appeared properly in the database navigator.







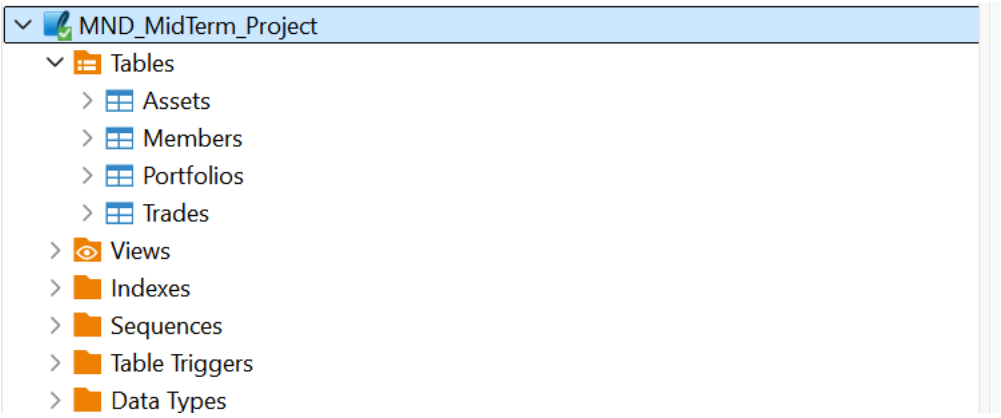


2. schema.sql – Table creation

To create the tables, I opened a new SQL editor in DBeaver by right-clicking on the "MND_MidTerm_Project" database, then executed the table creation script with PRAGMA foreign_keys = ON; at the top.

```
PRAGMA foreign_keys = ON;
-- ======================
-- MEMBERS
-- ======================
CREATE TABLE IF NOT EXISTS Members (
  member_id   INTEGER PRIMARY KEY,    -- Row Number from CSV (1..40)
  first_name  TEXT      NOT NULL,
  last_name   TEXT      NOT NULL,
  major       TEXT,
  join_date   TEXT                    -- 'YYYY-MM-DD'
);
-- ======================
-- PORTFOLIOS
-- ======================
CREATE TABLE IF NOT EXISTS Portfolios (
  portfolio_id   INTEGER PRIMARY KEY,  -- Row Number (1..60)
  member_id      INTEGER   NOT NULL,   -- FK -> Members
  portfolio_name TEXT      NOT NULL,
  created_date   TEXT,
  notes          TEXT,
  FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
CREATE INDEX IF NOT EXISTS idx_portfolios_member_id
  ON Portfolios(member_id);
-- ======================
-- ASSETS
-- ======================
CREATE TABLE IF NOT EXISTS Assets (
  asset_id     INTEGER PRIMARY KEY,   -- Row Number (1..120)
  symbol       TEXT    NOT NULL UNIQUE,
  asset_type   TEXT    NOT NULL,      -- STOCK / CRYPTO / FUND
  market       TEXT    NOT NULL,      -- NASDAQ/NYSE/ETF/CRYPTO
  current_price REAL
);
-- ======================
-- TRADES
-- ======================
CREATE TABLE IF NOT EXISTS Trades (
  trade_id    INTEGER PRIMARY KEY,    -- Row Number (1..800)
  portfolio_id INTEGER  NOT NULL,     -- FK -> Portfolios
  asset_id    INTEGER  NOT NULL,      -- FK -> Assets
  trade_date  TEXT     NOT NULL,      -- 'YYYY-MM-DD'
  trade_type  TEXT     NOT NULL,      -- BUY or SELL
  quantity    REAL     NOT NULL,      -- ints in our CSV (REAL is OK)
  trade_price REAL     NOT NULL,
  fee         REAL,                   -- may be NULL (~10%)
  notes       TEXT,
  FOREIGN KEY (portfolio_id) REFERENCES Portfolios(portfolio_id),
  FOREIGN KEY (asset_id)   REFERENCES Assets(asset_id)
);
```

After running the script (Ctrl+Enter), the tables were successfully generated and became visible in the navigator.



3. Quick data quality checks

We then proceeded with a few quick data quality checks by executing the following SQL script. These queries help ensure that the data was correctly imported, that primary keys are unique, and that relationships between tables are consistent.

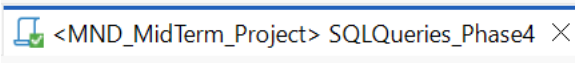
```
PRAGMA foreign_keys = ON;
/* 1) Row counts (should match your CSVs) */
SELECT (SELECT COUNT(*) FROM Members)   AS n_members,
    (SELECT COUNT(*) FROM Portfolios) AS n_portfolios,
    (SELECT COUNT(*) FROM Assets)    AS n_assets,
    (SELECT COUNT(*) FROM Trades)    AS n_trades;

/* 2) Orphan checks (each should return 0 rows) */
-- Portfolios without a valid Member
SELECT p.portfolio_id
FROM Portfolios p
LEFT JOIN Members m ON m.member_id = p.member_id
WHERE m.member_id IS NULL;

--- Trades without a valid Portfolio
SELECT t.trade_id
FROM Trades t
LEFT JOIN Portfolios p ON p.portfolio_id = t.portfolio_id
WHERE p.portfolio_id IS NULL;

--- Trades without a valid Asset
SELECT t.trade_id
FROM Trades t
LEFT JOIN Assets a ON a.asset_id = t.asset_id
WHERE a.asset_id IS NULL;
```

C. Phase 4 – Business analysis (queries)

Cf 

D. Reflection

Designing the BIC schema required me to translate narrative requirements into structured tables, keys, and cardinalities, and to maintain consistency throughout data generation and SQL implementation. I learned the importance of: (1) clear primary and foreign key design (using row number IDs in parent tables and bounded ranges in child tables), (2) respecting import order in SQLite/DBeaver (parents before children), and (3) adding quick "orphan" checks to validate referential integrity. On the query side, I practiced grouping, joins, CTEs, and precise filtering to ensure results were meaningful—not just technically correct.

How AI supported me

AI helped me correct my SQL schema to align with the ERD I had initially designed. It also suggested Mockaroo field settings—such as enums, ranges, and formulas—including a list of 120 US stock symbols, cryptocurrencies, and ETFs/funds. Additionally, it generated starter queries that matched the project rubric and helped me troubleshoot errors by proposing diagnostic techniques (e.g., orphan checks and integrity validations).

Where AI fell short (and how I corrected)

1. **Mockaroo formula language**: Some suggestions assumed JavaScript, while Mockaroo uses Ruby. I corrected this by writing and testing Ruby-based formulas directly in Mockaroo, iterating until I achieved the expected output.

2. **Label mismatch in generated data**: My trade_type field was generated as BUY (60%) / SELL (40%) using a weighted list, but my queries expected exact matches to 'BUY' and 'SELL'. I normalized the values using:

- UPDATE Trades SET trade_type = 'BUY' WHERE trade_type LIKE 'BUY%';
- UPDATE Trades SET trade_type = 'SELL' WHERE trade_type LIKE 'SELL%';

3. **Query alias and filters**: I had an alias typo (p vs pos) and a filter that was too strict (qty_held > 0 AND avg_cost IS NOT NULL), which returned empty results. I corrected the alias and used a more flexible filter (e.g., qty_held <> 0 to allow short positions) or rolled up results at the club level.

Prompts I used (samples + validation)

- I used ChatGPT to verify and validate my ERD design against my assumptions by tracing each business rule.

- "Draft SQLite DDL for the ERD (with PRAGMA foreign_keys)."

- "Mockaroo field settings for Assets/Trades with enums and a market formula." → Adjusted to Ruby syntax and previewed.

What I'd change next time

(1) Lock Mockaroo outputs with clean enums from the start to avoid post-import cleanup. (2) Add a short data-validation checklist before importing (e.g., unique symbols, valid enum values, date ranges). (3) Maintain small data_cleanup.sql and quality_checks.sql scripts for reproducibility. (4) Create views (e.g., Trades_normalized) to keep analysis queries simple and robust.